

UNIVERSITÉ PARIS CITÉ
UFR MATHÉMATIQUES ET INFORMATIQUE

Rapport de Projet : LLM - Automatic Argumentation Analysis

Master 2 Intelligence Artificielle Distribuée

Nassim LATTAB & Mohamed AZZAOU
Encadré par Benoît CRABBÉ

Année universitaire 2024-2025

Table des matières

1	Introduction	1
1.1	Contexte et Enjeux du Projet	1
1.2	Objectifs et Contributions	1
1.3	Organisation du Rapport	1
2	Approche Naïve pour l'Analyse de l'Argumentation	1
2.1	Construction du Vocabulaire et des Étiquettes BIO	1
2.2	Implémentation d'un Tokenizer Naïf	2
2.3	Création du Dataset pour PyTorch	2
2.4	Conception d'un Modèle Naïf pour le Tagging BIO	2
2.5	Entraînement avec Validation	2
2.6	Avantages et Limites Identifiés	3
3	Modèle Avancé : Exploitation d'un Large Language Model (LLM)	3
3.1	Choix du Modèle et Justifications	3
3.2	Fine-tuning pour la Détection d'Arguments	3
3.3	Pipeline et Intégration Technique	3
3.4	Avantages et Limites Identifiés (Approche LLM)	4
4	Analyse des Résultats	4
4.1	Métriques et Protocoles d'Évaluation	4
4.2	Comparaison Quantitative	4
4.3	Interprétation et Analyse Qualitative	5
5	Difficultés Rencontrées	5
6	Conclusion et Perspectives	5
6.1	Synthèse des Travaux	5
6.2	Impacts Potentiels	6
6.3	Perspectives d'Évolution	6

1 Introduction

1.1 Contexte et Enjeux du Projet

L’argumentation, déjà largement étudiée dans d’autres disciplines, joue un rôle central dans l’analyse critique et la communication. Dans ce projet, nous prolongeons cette réflexion en l’appliquant au *traitement du langage naturel* (NLP). Nous comparons deux approches de **matching d’arguments** :

- une méthode **naïve**, simple et peu gourmande en ressources,
- un **modèle basé sur les grands modèles de langage (LLM)**, capable de mieux saisir les nuances contextuelles.

Ces solutions sont évaluées sur des jeux de données issus de domaines variés (essais, textes médicaux), afin de mesurer leur efficacité et leur capacité à généraliser. Le présent rapport décrit en détail la méthodologie mise en place, les défis rencontrés et les solutions adoptées.

1.2 Objectifs et Contributions

L’objectif principal de ce travail est de développer un système capable d’**identifier** et de **caractériser** les segments argumentatifs dans des textes, quel que soit leur domaine. Plus précisément, nous visons à :

- **Concevoir et comparer plusieurs méthodes** d’argument matching, depuis la plus simple jusqu’à la plus sophistiquée, afin de mesurer l’impact du type de modèle sur la précision des prédictions.
- **Évaluer la robustesse** de ces approches sur différents jeux de données, pour juger leur capacité de généralisation.

Sur le plan méthodologique, notre contribution réside dans la mise en place d’un **pipeline flexible et modulaire** permettant l’implémentation et le test de diverses stratégies. Enfin, nous proposons un **comparatif détaillé** des performances et limites de chaque approche, accompagné de pistes d’amélioration.

1.3 Organisation du Rapport

Le rapport est structuré en sept grandes parties. Après cette introduction, la Section 2 se penche sur l’approche naïve pour l’analyse de l’argumentation : nous y décrivons la construction du vocabulaire, des **étiquettes BIO**, le *tokenizer* et le *dataset*, avant de présenter le modèle à base de LSTM et d’en analyser les performances. La Section 3 introduit l’approche avancée, un **Large Language Model** (RoBERTa), dont nous exposons le choix, le *fine-tuning* spécifique à la détection d’arguments et le **pipeline** technique. La Section 4 analyse les résultats obtenus avec le modèle naïf et le LLM, en détaillant les métriques utilisées, l’évaluation stricte ou relaxée, ainsi que les erreurs types. La Section 5 aborde les difficultés rencontrées (infrastructure de calcul, manque de données médicales) et les solutions envisagées. Enfin, la Section 6 conclut sur les enseignements tirés du projet et ouvre sur les perspectives d’évolution des LLM et l’optimisation des hyperparamètres.

2 Approche Naïve pour l’Analyse de l’Argumentation

2.1 Construction du Vocabulaire et des Étiquettes BIO

Dans un premier temps, nous avons implémenté la fonction `build_vocab_and_labels(data)`, dont le rôle est d’extraire à partir des données d’entraînement :

- un **vocabulaire**, *i.e.* l’ensemble des mots uniques présents dans le corpus,

- des **étiquettes BIO**, indiquant pour chaque mot s'il se situe en **B-Claim** (début d'argument), **I-Premise** (suite d'argument) ou **O** (hors argument).

Pour mieux gérer les mots inconnus et les séquences de longueur variable lors de l'entraînement, nous avons ajouté deux *tokens* spéciaux : **<unk>** pour les mots inconnus et **<pad>** pour le remplissage (*padding*).

2.2 Implémentation d'un Tokenizer Naïf

Nous avons développé une classe `NaiveTokenizer` dont l'objectif est de convertir les mots et leurs **étiquettes BIO** en **indices numériques** manipulables par le modèle. Cette classe :

- **Encode** une séquence de mots en indices numériques,
- **Décode** ces indices pour retrouver la forme textuelle originale,
- **Convertit** les étiquettes BIO en indices spécifiques,
- **Gère le *padding*** des séquences plus courtes à l'aide du *token* **<pad>**,
- **Regroupe en *batches*** les données nécessaires à l'entraînement.

2.3 Création du Dataset pour PyTorch

Pour structurer et rendre les données compatibles avec PyTorch, nous avons défini la classe `ArgumentationDataset`. Elle transforme les données brutes en un format exploitable par le modèle :

- Chaque document est découpé en paragraphes,
- Les mots sont encodés en indices via le `NaiveTokenizer`,
- Les étiquettes BIO sont également converties en indices.

Cette approche aboutit à un **dataset structuré**, favorisant l'apprentissage et l'évaluation sur des exemples cohérents d'argumentation.

2.4 Conception d'un Modèle Naïf pour le Tagging BIO

La suite du travail a consisté à développer le `NaiveBIOModel`, un modèle de **deep learning** pour le *tagging* de séquences. Son architecture inclut :

- **Une couche d'*embedding*** pour transformer chaque indice de mot en un vecteur dense,
- **Un LSTM (Long Short-Term Memory)** pour capturer le contexte et les relations entre mots dans une phrase,
- **Une couche linéaire** reliant la sortie du LSTM aux scores pour chaque étiquette BIO.

La fonction de perte (`CrossEntropyLoss` dans la plupart des cas) calcule l'écart entre les prédictions et les étiquettes réelles, guidant ainsi l'ajustement des poids pour améliorer la précision.

2.5 Entraînement avec Validation

Pour entraîner notre modèle et éviter le *surapprentissage*, nous avons suivi une procédure en plusieurs étapes. Nous parcourons d'abord l'ensemble des données et les regroupons sous forme de ***batches*** pour optimiser le traitement en parallèle. Lors de la **propagation avant**, le modèle prédit les étiquettes BIO pour chaque séquence. Une fonction de perte compare ensuite ces prédictions avec les annotations réelles et calcule un **score d'erreur**, qui alimente la **rétropropagation** et ajuste les poids du réseau. Pour contrôler la performance et limiter le **surapprentissage**, nous évaluons régulièrement le modèle sur un jeu de **validation** après chaque époque. En cas de dégradation des résultats ou de signes d'*overfitting*, nous adaptons divers hyperparamètres (taux d'apprentissage, taille des *batches*, etc.) pour améliorer la généralisation.

2.6 Avantages et Limites Identifiés

Le principal atout de cette approche réside dans sa **simplicité** de mise en œuvre et sa **rapidité** d'entraînement, avec un besoin en ressources nettement moindre qu'un **LLM**. Malgré cela, le modèle atteint un **taux de réussite d'environ 60%**, ce qui demeure raisonnable pour une solution minimaliste. Toutefois, il se heurte vite à un **plafond de performance**, rendant difficile toute amélioration substantielle. Pour dépasser ces limites et capturer davantage de nuances linguistiques, le recours à une architecture LLM s'impose.

3 Modèle Avancé : Exploitation d'un Large Language Model (LLM)

3.1 Choix du Modèle et Justifications

Dans ce projet, nous avons choisi **RoBERTa**, un modèle de type *Transformer* pré-entraîné. Par rapport à BERT, RoBERTa profite d'un pré-entraînement plus poussé (masquage dynamique, corpus plus volumineux, etc.), ce qui améliore ses performances sur des tâches de **tagging de séquence** et de classification au niveau du *token*. RoBERTa se révèle plus robuste que DistilBERT (mais plus exigeant en ressources) et s'adapte mieux qu'un GPT lorsqu'on souhaite attribuer des étiquettes supervisées plutôt que générer du texte. Ce **compromis entre puissance et efficacité** en fait un candidat pertinent pour l'analyse argumentative, où il faut détecter précisément des segments (ou *spans*) et les annoter selon leur rôle (claim, premise, etc.).

3.2 Fine-tuning pour la Détection d'Arguments

Pour spécialiser RoBERTa à la tâche de détection d'arguments, nous avons réalisé un **fine-tuning** analogue à une tâche de **Named Entity Recognition (NER)**. Les grandes étapes sont :

- **Spécifier la sortie** : ajout d'une couche linéaire en sortie de RoBERTa, pour prédire un label par *token* (format BIO).
- **Gérer les tags spéciaux** : définition d'une classe de *padding* (ID distinct) à ignorer dans la fonction de perte.
- **Paramétrer l'entraînement** :
 - Fonction de perte : **cross-entropy**, ignorant les *tokens* de remplissage,
 - **Scheduler** : *ReduceLROnPlateau*, réduisant le taux d'apprentissage en cas de stagnation de la validation,
 - **Early stopping** : arrêt de l'entraînement si la performance de validation n'évolue plus après un certain nombre d'époques.

Concrètement, la classe `RobertaTagger` (héritant de `nn.Module`) charge `roberta-base` de FacebookAI et y ajoute un classifieur linéaire adapté au nombre d'étiquettes (BIO + rôles argumentatifs).

3.3 Pipeline et Intégration Technique

Pour l'intégration, nous avons défini un **pipeline complet** s'appuyant sur `transformers` et `PyTorch` :

- **Chargement et prétraitement des données** : Les corpus (fichiers JSON) sont chargés, et les textes/annotations alignés pour la *tokenisation* RoBERTa.
- **Tokenisation et Construction du Dataset** : Le tokenizer RoBERTa segmente les phrases en *subwords*, et les étiquettes correspondantes sont mappées. Les labels hors limites ou inutilisés sont remplacés par le *tag* de PAD. Nous définissons une classe `Dataset` PyTorch pour retourner `input_ids` et `labels` par *batch*.

- **Initialisation du Modèle et Entraînement** : `RobertaTagger` est instancié, et RoBERTa est *fine-tuné* sur nos données avec un **taux d'apprentissage modéré**. Une **validation** a lieu à chaque époque, avec **early stopping** pour limiter le *surapprentissage*.
- **Validation et Prédiction** : À chaque époque, la perte est calculée sur le jeu de validation. Quand elle n'évolue plus pendant plusieurs itérations, l'entraînement s'arrête. En phase de test, le modèle prédit les labels pour chaque *token*, permettant de reconstituer les segments argumentatifs pour l'évaluation finale.

Ce pipeline facilite la **détection d'arguments** de bout en bout : du chargement des textes à la reconstruction finale des *spans*. Des optimisations comme le *batching* et le **scheduler** améliorent la performance tout en maîtrisant le temps de calcul.

3.4 Avantages et Limites Identifiés (Approche LLM)

L'**utilisation d'un LLM** pour l'extraction et la classification textuelle présente des avantages notables. D'abord, le pré-entraînement massif sur des corpus variés confère au modèle **une compréhension fine des nuances linguistiques**, atout majeur pour des tâches complexes comme la détection de *spans* argumentatifs. De plus, sa **flexibilité** lui permet de s'adapter facilement (via **fine-tuning**) à des domaines spécifiques (juridique, médical, politique), pour peu que l'on dispose d'un **jeu de données annoté** suffisant.

En contrepartie, le **coût en ressources** est plus élevé : le fine-tuning et l'inférence sur un LLM requièrent souvent des GPU ou TPU puissants, ce qui peut constituer un frein économique et technique. L'**implémentation** se révèle aussi plus complexe : l'ajustement du *scheduler*, de l'*early stopping* et des évaluations successives suppose une **expertise** en ingénierie de modèles pour gérer le *surapprentissage* et stabiliser la formation.

En résumé, un LLM offre **une précision** et **une adaptabilité** nettement supérieures, au prix d'une **infrastructure matérielle** plus lourde et d'un pilotage de projet exigeant.

4 Analyse des Résultats

Cette section propose une comparaison générale des performances entre le **modèle naïf** et le **modèle LLM** (RoBERTa affiné) sur le corpus AAE. Le corpus médical n'est pas inclus dans l'évaluation finale, son jeu de données étant trop restreint.

4.1 Métriques et Protocoles d'Évaluation

Nous avons utilisé la **précision**, le **rappel** et le **F1-score** pour mesurer la capacité de chaque modèle à repérer et classer correctement les segments argumentatifs. Deux modalités d'évaluation ont été retenues :

- **Évaluation stricte** : les bornes du segment prédit doivent coïncider exactement avec celles de la référence.
- **Évaluation relaxée** ($\alpha = 0.5$) : le segment prédit est jugé correct s'il recouvre au moins 50% du segment de référence.

Par ailleurs, nous avons différencié l'**identification pure** (*unlabeled*) de l'**identification avec catégorie** (*labeled*). Un script `evaluate.py` a été utilisé pour automatiser ces calculs et assurer une **comparaison homogène** entre approches.

4.2 Comparaison Quantitative

Modèle naïf. En **évaluation stricte**, les scores de F1 sont relativement modestes : le modèle détecte une partie des segments, mais peine à aligner précisément leurs limites. En **mode relaxé**,

la performance s’améliore, illustrant une tendance à « cerner » la bonne zone argumentative sans toujours marquer les bornes exactes.

Modèle LLM (RoBERTa Fine-Tuné). Il surpasse largement le modèle naïf, avec une **amélioration notable de la précision et du rappel**, et donc un F1-score supérieur. L’écart se creuse surtout en évaluation **stricte**, signe que RoBERTa localise mieux le début et la fin de chaque segment. En mode relaxé, le modèle confirme sa capacité à « bien cerner » le contenu argumentatif.

4.3 Interprétation et Analyse Qualitative

Erreurs fréquentes.

- **Faux positifs** : le modèle naïf confond parfois des phrases de contexte avec des arguments. Le LLM, plus fin, réduit ces confusions, même s’il peut encore parfois englober plusieurs arguments en un segment trop long.
- **Faux négatifs** : le modèle naïf omet certains arguments brefs ou implicites. Le LLM les reconnaît plus souvent, mais peut rester en échec sur des phrases mêlant plusieurs fonctions argumentatives (affirmation, contre-argument, justification).

Comportement d’entraînement. Le *log* montre que le LLM converge rapidement vers un **niveau de perte satisfaisant** avant de plafonner, moment où intervient l’*early stopping*. Le modèle naïf, lui, stagne plus tôt et peine à réduire davantage la **perte de validation**.

Finalement, le **LLM** offre **des gains substantiels** en précision, rappel et F1-score. Il est toutefois possible d’envisager des optimisations complémentaires (augmentation de la durée d’entraînement, meilleur découpage des données, enrichissement des corpus), mais les résultats actuels suggèrent déjà l’intérêt de l’**affinage d’un modèle pré-entraîné** pour la détection et la classification d’arguments.

5 Difficultés Rencontrées

Nous avons initialement planifié d’entraîner RoBERTa sur **Google Colab**, mais les temps de calcul sont rapidement devenus prohibitifs, surtout lors du réglage d’hyperparamètres. Nous avons donc migré vers une machine locale équipée d’un **GPU NVIDIA** et exploitant **CUDA**. Cette transition a **accéléré l’entraînement** et offert un contrôle accru sur l’infrastructure et la configuration logicielle.

Par ailleurs, le **jeu de données médical** s’est avéré trop limité, menant à des performances décevantes et un risque marqué de *surapprentissage*. Pour y remédier, nous prévoyons de **fusionner plusieurs jeux de données médicaux** afin de constituer un ensemble plus vaste et représentatif de la diversité clinique. Nous espérons ainsi **renforcer la robustesse** et la capacité de **généralisation** du modèle (extraction d’informations, aide au diagnostic, etc.).

6 Conclusion et Perspectives

6.1 Synthèse des Travaux

Dans ce projet, nous avons expérimenté deux approches d’analyse argumentative. Le **modèle naïf**, d’abord, a facilité la mise en place des principes de **détection** et de **classification** des segments, tout en dévoilant assez vite les limites d’une architecture rudimentaire dépourvue de représentation linguistique poussée. Nous sommes ensuite passés à un **LLM (RoBERTa)**, déjà **pré-entraîné**, qui a montré **des progrès significatifs** sur la détection et la catégorisation des arguments. Au-delà d’un simple saut dans les métriques, l’intégration d’un contexte

lexical et syntaxique plus riche a permis de mieux appréhender les structures argumentatives complexes.

6.2 Impacts Potentiels

Les applications de l’analyse argumentative **automatisée** sont nombreuses :

- **Analyse de débats et de discours publics** : politiques, chercheurs en sciences sociales ou médias peuvent y recourir pour étudier la structure et la rhétorique des discours.
- **Revue de littérature scientifique** : un outil détectant rapidement arguments et prémisses dans un large ensemble de publications faciliterait la synthèse et la veille.
- **Modération et compréhension sur les réseaux sociaux** : distinguer les messages réellement argumentés de ceux qui sont fallacieux ou hors sujet peut aider les plateformes à repérer et mettre en avant des contributions pertinentes.
- **Assistance à la rédaction et au débat** : des éditeurs de texte ou des outils collaboratifs pourraient proposer un support pour la structuration argumentative en temps réel.

Au niveau industriel, cette capacité à **extraire et analyser automatiquement** la structure argumentative est prisée par de nombreux acteurs (veille, *sentiment analysis*, modération). Dans le milieu académique, elle intéresse la **rhétorique**, la **linguistique computationnelle** et la **psychologie cognitive**.

6.3 Perspectives d’Évolution

Plusieurs pistes s’ouvrent pour améliorer notre système d’analyse argumentative :

- **Adopter un LLM plus performant** : Modèles de plus grande taille (GPT- x) ou variantes récentes. Ce choix requiert toutefois **plus de puissance de calcul** et de mémoire GPU.
- **Enrichir les jeux de données** : Combiner diverses sources pour accroître la diversité et la **capacité de généralisation**.
- **Optimiser les hyperparamètres** : Recours à des bibliothèques telles que GridSearchCV de scikit-learn ou à des alternatives plus avancées comme Optuna pour une recherche plus efficace des hyperparamètres optimaux (nombre d’époques, batch size, learning rate, etc...).
- **Affiner la stratégie d’entraînement** : Ajuster le nombre d’époques et l’*early stopping* pour éviter le sous-apprentissage,

En somme, ces améliorations — qu’il s’agisse d’augmenter la taille des modèles, de diversifier les données ou de peaufiner l’optimisation — pourraient **consolider** à la fois la **fiabilité** et la **portée** de ce système dédié à l’analyse automatisée de l’argumentation.